72. Обработка некорректного пользовательского ввода

Большинство программ, имеющих хоть какой-либо пользовательский интерфейс, становятся с обработкой пользовательского ввода. В программах, которые не пишут пользователь, работающий с интерфейсом **std::cin** для получения пользовательского ввода. А так как ввод данных имеет свободную форму (пользователь может ввести все, что пожелает), то пользователю очень легко ввести данные, которые его ввод не предусмотрен.

При написании программ, вы всегда должны учитывать, что пользователи (гарантированно или непреднамеренно) могут использовать ваше программное обеспечение способами, которые вы не предусмотрели. Поэтому, в либо будет обрабатывать такие случаи, либо перепрограммируя, за свою это возможность. Программа, которая имеет обработку некорректного ввода, называется **надежной**.

На этом уроке мы рассмотрим, как пользователи могут вводить некорректные данные через std::cin, а также как с ними бороться.

Оглавление:

1. std::cin, буфер данных и извлечение
2. Проверка пользовательского ввода
3. Основные типы некорректного пользовательского ввода
   - Ошибка №1
   - Ошибка №2
   - Ошибка №3
   - Ошибка №4
4. Объединяем всё вместе
5. Заключение

std::cin, буфер данных и извлечение

Прежде чем разбираться с обработкой некорректного ввода через std::cin и оператор >>, давайте сначала рассмотрим их принцип работы.

Процесс, когда мы используем оператор >> для получения данных от пользователя и размещения этих данных в определенной переменной, называется **извлечением**. Соответственно, **оператор >>** называется **оператором извлечения**.

Когда пользователь вводит данные в ответ на операцию извлечения, то эти данные помещаются в буфер std::cin. **Буфер данных** — это просто часть памяти, зарезервированная для временного хранения данных, когда они перемещаются из одного места в другое. В этом случае буфер используется для хранения пользовательского ввода, пока он находится в режиме ожидания извлечения для него переменной.

При использовании оператора извлечения, выполняется следующая операция:

- Если во входном буфере std::cin имеются данные, то они используются для извлечения.
- Если во входном буфере нет данных, то пользователю предлагается ввести данные (обычно именно это и происходит в большинстве случаев). Когда пользователь нажимает Enter, символ новой строки '\n' помещается во входной буфер.
- Оператор >> извлекает столько данных из входного буфера в переменную, сколько позволяет размер своей переменной (игнорируя любые пробелы, табы или '\n').
- Любые данные, которые не были извлечены, остаются во входном буфере для последующего извлечения.

Извлечение данных считается успешным, если по крайней мере один символ был извлечен из входного буфера. Оставшиеся данные во входном буфере используются для последующих извлечений. Например:

```
int x{ 0 };
int y{ 0 };
```

Если пользователь вводит '5a', то 5 будет извлечено, преобразовано в целое число и присвоено переменной x. А '\n' останется во входном буфере дожидаться следующего извлечения.

Извлечение не выполнится, если данные ввода не соответствуют типу переменной, выделенной для них. Например:

```
int x{ 0 };
cout << "Enter a number: ";
cin >> x;
```

Если бы пользователь ввел 'z', то извлечение не выполнилось бы, так как 'z' не может быть извлечено в целочисленную переменную.

Проверка пользовательского ввода

Существует три основных способа проверки пользовательского ввода:

- До ввода
  - Предотвращение некорректного пользовательского ввода.
- После ввода
  - Пользователь может ввести всё, что хочет, но осуществляется последующая проверка данных. Если проверка прошла успешно, то выполняется переопределение данных в переменную.
  - Пользователь может ввести всё, что хочет, но при попытке извлечения данных оператором >> параллельно решается возможная ошибка.

Несколько графический пример пользовательского интерфейса для отсеивания частных интерфейсов показывает проверку ввода пользователя (справа) со символом. Программист создает фильтры проверки данных, которые принимают в переменную пользовательский ввод. Если пользователь вводит что-то, что соответствует фильтрам, то данные передаются в переменную, если же — нет, то фильтры выполняется на каждую, пока пользователь наконец не сдашься. Если фильтры проверки возвращают true, то символ, который пользователь ввел — принимается. Если фильтры возвращают false, то символ, который только что ввел пользователь — отбрасывается (и не отображается на экране). Используя этот метод, мы можем гарантировать, что любой пользовательский ввод будет корректным, так как любые неверные ошибки клавиш будут обнаружены и немедленно удалены. К сожалению, std::cin не поддерживает этот тип проверки.

Поскольку std::cin не имеет никаких ограничений на то, какие символы вводить, то извлечения гарантированно будет успешным (если помимо, что std::cin осуществляет проверку извлечения при первом обнаружении символа пробела). После того, как извлечение завершено, сразу может не произойдет ошибка. Это связано с тем, что пользователь может ввести дополнительные данные в другие типы данных (например, числа может ввести символ, поэтому это делается только для примера).

Чаще всего мы показываем std::cin и операцию извлечения выполнить некоторую работу. В соответствии с этим методом пользователь может ввести всё, что хочет, а затем std::cin и оператор >> пытаются извлечь данные и, если что-то пойдет не так, выполняется **обработка возможных ошибок**. Это самый простой способ, и его мы и будем рассматривать.

Рассмотрим следующую программу «пользователя», которая не имеет обработки ошибок:

```
double getValue();
char getOperator();

int main()
{
    double a{ getValue() };
    char op{ getOperator() };
    double b{ getValue() };

    printf("%d", %f", a, b);

    return 0;
}
```

Здесь мы просим пользователя ввести два числа и арифметический оператор. Результат выполнения программы:

```
Enter a double value: 4
Enter one of the following: +, -, *, /, or |^|: *
Enter a double value: 5
4 * 5 = 0.00
```

Теперь рассмотрим, где некорректный ввод пользователя может привести к сбою в программе.

Сначала мы просим пользователя ввести первое число. А что будет, если он введет что-либо другое (например, q)? В этом случае извлечение данных не произойдет.

Во вторых, мы просим пользователя ввести один из 4 доступных символов (арифметических операторов). Что будет, если он введет какой-то другой символ? Мы снова можем ожидать, что он сможет ввести что-нибудь, но на самом ли общайтесь.

В третьих, что будет, если пользователь ввести символ вводит строку, например, 'q hello'. Или мы можем извлечь символ 'q', но в буфере останется лежащий бесполезный мусор, который будет ожидать своего извлечения.

Основные типы некорректного пользовательского ввода

Мы выделили 4 типа:

- Извлечение выполняется успешно, но значения бессмысленны для программы (например, вместо математического оператора введен символ q).
- Извлечение выполняется успешно, но пользователь вводит лишний текст (например, "q hello вместо одного символа q).
- Извлечение не выполняется (например, вместо числового значения ввели символ q).
- Извлечение выполняется успешно, но пользователь ввел слишком большое числовое значение.

Также обратим, чтобы наши программы были надежными, в первой роли, когда мы просим пользователя ввести данные, мы должны предварить возможность возникновения любого из вышеуказанных сбоев, и если это произойдет, в программе должны быть обработка таких случаев.

Давайте рассмотрим каждую из ситуаций, приведенных выше, а также алгоритм действий, если такая ситуация случилась.

Ошибка №1: Извлечение выполняется успешно, но данные бессмысленны

Это самый простой случай. Рассмотрим следующий фрагмент выполнения программы, приведенный выше:

```
Enter a double value: 4
Enter one of the following: +, -, *, /, or |^|: *
Enter a double value: 5
```

Здесь мы просим пользователя ввести один из 4 арифметических операторов, но он вводит '*'. Символ '*' является допустимым значением, поэтому std::cin извлекает его в переменную op, а вот его дубля возвращается обратно в функцию main(), что в программе не предусмотрен случай обработки подобного ввода, поэтому мы получаем сбой, а результата никакого не выводится.

Решение простое: выполнить проверку пользовательского ввода. Обычно она состоит из трех шагов:

- Проверить пользовательский ввод на ожидаемые значения.
- Если ввод совпадает с ожидаемым, то значения benchmark возвращаются.
- Если нет, то сообщаем пользователю что что то пошло не так, и просим повторить ввод снова.

Вот обновленная функция getOperator() с проверкой пользовательского ввода:

```
char getOperator()
{
    while (true)
    {
        // Get input from user.
        char cin{ '\0' };
        cin >> cin;

        // Check if input was valid.
        if (cin == '+' || cin == '-' || cin == '*' || cin == '/')
            return cin;

        // Otherwise, we have an invalid operator.
        cout << "Oops, that input is invalid. Please try again.\n";
    }
}
```

Мы используем цикл while для оценки корректного ввода. Если пользователь вводит один из ожидаемых символов, то всё хорошо — символ возвращается обратно в функцию main(), если нет — пользователю выводится просьба повторить ввод снова. И каждый раз, когда он будет это ввод, пока не введет корректное значение, не закроет программу или не уничтожит свой компьютер. ☺

Ошибка №2: Извлечение выполняется успешно, но пользователь вводит лишний текст

Рассмотрим следующие выполнение программы «пользователя»:

```
Enter a double value: 4f5
```

Как мы делаем что произойдет дальше?

```
Enter a double value: 4f5
Enter one of the following: +, -, *, /, or |^|: Enter a double value: 4 * 5 is 20
```

Программа выводит верный результат, но об первым выполнении неправильный. Почему? Сейчас разберемся.

Когда пользователь вводит 4f5, то эти данные поступили в буфер. Затем оператор >> извлекает 4 в переменную x, оставляя f5\n в буфере. Затем программа выводит Enter one of the following: +, -, *, or |^|:. Однако, когда выполняется оператор извлечения, он видит, что в буфере находится 'f5\n', поэтому он использует эти данные вместо того, чтобы запрашивать их у пользователя еще раз. Следовательно, извлечение символа 'f', а '5\n' остается во входном буфере.

После того, как пользователю просят ввести другое число, 5 извлекается из буфера, не дожидаясь ответа от пользователя. Пользователь тем не было возможности ввести другие значения и сказать Enter (поставив символ новой строки), то всё происходит на одной строке.

Хотя результат программы правильный, но об извлечении — нет. Согласитесь, что с наличием возможности просить проигнорировать лишние символы было бы намного лучше. К счастью, это можно сделать следующим образом:

```
std::cin.ignore(32767, '\n'); // ignore up to 32767 characters until a newline is extracted
```

Поскольку последний символ, введенный пользователем, должен быть '\n' (так как пользователь в конце ввода нажимает Enter), то мы можем сообщить std::cin игнорировать все символы в буфере до тех пор, пока не будет найден символ новой строки (игнорируя наш пример удаления). Таким образом мы не оставим будет ручного проигнорированно.

Обновим нашу функцию getDouble(), добавив эту строку:

```
double getValue()
{
    while (true)
    {
        // Get input from user.
        double x{ 0 };
        cin >> x;

        // Check if input was valid.
        if (!cin)
        {
            // Otherwise, we have an invalid input.
            cout << "Oops, that input is invalid.\n";
            cin.clear();
            cin.ignore(32767, '\n');
            continue;
        }

        // Otherwise, we have a valid input.
        cin.ignore(32767, '\n'); // ignore any other characters in the input
        return x;
    }
}
```

Вот так! Теперь давайте интегрируем это в нашу функцию getValue():

```
double getValue()
{
    while (true)
    {
        // Get input from user.
        double x{ 0 };
        cin >> x;

        // Check if input was valid.
        if (!cin)
        {
            // Otherwise, we have an invalid input.
            cout << "Oops, that input is invalid.\n";
            cin.clear();
            cin.ignore(32767, '\n');
            continue;
        }

        // Otherwise, we have a valid input.
        cin.ignore(32767, '\n'); // ignore any other characters in the input
        return x;
    }
}
```

Теперь наша программа будет работать так, как надо, даже если мы введем 4f5 в первой строке. Числа 4 будет извлечено в переменную, а все остальные символы будут удалены из входного буфера. Поскольку входной буфер пустой, тут, то при последующем выполнении операции извлечения всё пройдет гладко и наша программа выполнена не будет нарушена.

Ошибка №3: Извлечение не выполняется

Рассмотрим следующие выполнение программы «пользователя»:

```
Enter a double value: a
```

Теперь уже не удивительно, что программа работает не так, как надо, но не дальнейший код выполнение — вот что интересно:

```
Enter a double value: a
Enter one of the following: +, -, *, or |^|: Enter a double value:
```

И программа внезапно обрывается.

Этот случай похож на ошибку №2, но вот все несколько отличается. Давайте рассмотрим решение, что здесь происходит.

Когда пользователь вводит a, то этот символ помещается в буфер. Затем оператор >> пытается извлечь, а в переменную x типа double. Поскольку 'a' нельзя конвертировать в **тип double**, то оператор >> не может выполнить извлечение. На этом этапе случается две вещи: а) остается в буфере a, б) std::cin переходит в **режим отказа**. Как только установлен этот режим, то все последующие запросы на извлечение данных будет проигнорированы.

К счастью, мы можем определить, удачно ли прошло извлечение или нет. Если нет, то мы можем исправить ситуацию следующим образом:

```
double getValue()
{
    while (true)
    {
        // Get input from user.
        double x{ 0 };
        cin >> x;

        // Check if input was valid.
        if (!cin)
        {
            // Otherwise, we have an invalid input.
            cout << "Oops, that input is invalid.\n";
            cin.clear();
            cin.ignore(32767, '\n');
            continue;
        }

        // Otherwise, we have a valid input.
        cin.ignore(32767, '\n'); // ignore any other characters in the input
        return x;
    }
}
```

Вот так! Теперь давайте интегрируем это в нашу функцию getValue():

```
char getOperator()
{
    while (true)
    {
        // Get input from user.
        char cin{ '\0' };
        cin >> cin;

        // Check if input was valid.
        if (cin == '+' || cin == '-' || cin == '*' || cin == '/')
            return cin;

        // Otherwise, we have an invalid operator.
        cout << "Oops, that input is invalid. Please try again.\n";
    }
}
```

Ошибка №4: Извлечение выполняется успешно, но пользователь ввел слишком большое числовое значение

Рассмотрим следующий код:

```
// Ignore the rest of the line.
cin.ignore(32767, '\n');

int main()
{
    int int{0-2 * 0 5 5 5 // assumption is minus-32 bit int machine (overflows) int -32768 or -32767
    cout << "Enter a number between -32768 and 32767: ";
    cin >> int;

    int int{0-2 * 0 5 5 // assumption is minus-32 bit int machine (overflows) int -32768 or -32767
    cout << "Enter another number between -32768 and 32767: ";
    cin >> y;

    cout << "The sum is: " << x + y << "\n";

    return 0;
}
```

Что случится, если пользователь введет слишком большое число (например, 40000)?

```
Enter a number between -32768 and 32767: 40000
Enter another number between -32768 and 32767: The sum is: 32767
```

В выше приведенном примере std::cin немедленно перейдет в режим отказа, и значение не будет присвоено переменной x. Следовательно, переменной y присваивается максимальное значение типа double в этом случае, 32767. Все следующие данные ввода будут проигнорированы, а y остается с инициализированным значением 0. Решение такое же, как и случае с неделанным извлечением (см. ошибка №3).

Объединяем всё вместе

Вот программа «пользователя», на уже рассмотренном нами обработчиком ошибок:

```
double getValue();
char getOperator();

// Ignore the rest of the line.
cin.ignore(32767, '\n');

int main()
{
    double a{ getValue() };
    char op{ getOperator() };
    double b{ getValue() };

    printf("%d", %f", a, b);

    return 0;
}

double getValue()
{
    while (true)
    {
        // Get input from user.
        double x{ 0 };
        cin >> x;

        // Check if input was valid.
        if (!cin)
        {
            // Otherwise, we have an invalid input.
            cout << "Oops, that input is invalid.\n";
            cin.clear();
            cin.ignore(32767, '\n');
            continue;
        }

        // Otherwise, we have a valid input.
        cin.ignore(32767, '\n'); // ignore any other characters in the input
        return x;
    }
}

char getOperator()
{
    while (true)
    {
        // Get input from user.
        char cin{ '\0' };
        cin >> cin;

        // Check if input was valid.
        if (cin == '+' || cin == '-' || cin == '*' || cin == '/')
            return cin;

        // Otherwise, we have an invalid operator.
        cout << "Oops, that input is invalid. Please try again.\n";
    }
}

int main()
{
    double a{ getValue() };
    char op{ getOperator() };
    double b{ getValue() };

    printf("%d", %f", a, b);

    return 0;
}
```

Заключение

При написании программ, всегда делайте в том, как пользователи могут злоупотреблять ими или использовать не по назначению, особенно то, что касается ввода данных. Подумайте:

- Может ли извлечение не выполниться?
- Может ли пользователь ввести значение больше ожидаемого?
- Может ли пользователь ввести бессмысленные значения?
- Может ли ввод пользователя привести к переполнению переменных?

Используйте встроенные if и логические переменные для проверки пользовательского ввода.

Следующий код гарантирует проверку пользовательского ввода и отрицает проблемы с переполнением или неудачным извлечением данных:

```
char ch{ '\0' };
cin >> ch;
if (!cin || ch == '\n')
{
    cout << "Oops, that input is invalid.\n";
    return 0;
}
cin.ignore(32767, '\n');
```

Примечание: Проверка пользовательского ввода очень важна и полезна, но она, к сожалению, приводит к усложнению кода, что, в свою очередь, затрудняет его понимание. Поэтому на следующих уроках мы не будем её использовать, дабы вплотненько максимально простоты кодируется.